



National University
of computer and emerging sciences

Computer Organization and Assembly Language (EL-2003)

Semester: Fall 2025

Section: BCS-3D / BCS-3H

Course Instructor: Muhammad Owais

LAB # 04

Working with Operators, Instructions and Symbolics Constants

Lab Objectives:

By the end of this lab, students will be able to:

1. Assembly language Instructions: MOV , ADD , SUB , INC , DEC, MOVZX, MOVSX, XCHG
2. Some useful Assembly Language Operators DUP, EQU

Operand Types

As x86 instruction formats:

[label:] mnemonic [operands][; comment]

Because the number of operands may vary, we can further subdivide the formats to have zero, one, two, or three operands. Here, we omit the label and comment fields for clarity:

mnemonic

mnemonic [destination]

mnemonic [destination],[source]

mnemonic [destination],[source-1],[source-2]

x86 assembly language uses different types of instruction operands. The following are the easiest to use:

- Immediate—uses a numeric literal expression
- Register—uses a named register in the CPU

- Memory—references a memory location

Following table lists a simple notation for operands. We will use it from this point on to describe the syntax of individual instructions.

Operand	Description
reg8	8-bit general-purpose register: AH, AL, BH, BL, CH, CL, DH, DL
reg16	16-bit general-purpose register: AX, BX, CX, DX, SI, DI, SP, BP
reg32	32-bit general-purpose register: EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP
reg	Any general-purpose register
sreg	16-bit segment register: CS, DS, SS, ES, FS, GS
imm	8-, 16-, or 32-bit immediate value
imm8	8-bit immediate byte value
imm16	16-bit immediate word value
imm32	32-bit immediate doubleword value
reg/mem8	8-bit operand, which can be an 8-bit general register or memory byte
reg/mem16	16-bit operand, which can be a 16-bit general register or memory word
reg/mem32	32-bit operand, which can be a 32-bit general register or memory doubleword
mem	An 8-, 16-, or 32-bit memory operand

MOV Instruction

It is used to move data from source operand to destination operand

- Both operands must be the same size.
- Both operands cannot be memory operands.
- CS, EIP, and IP cannot be destination operands.
- An immediate value cannot be moved to a segment register.

Syntax:

MOV destination, source

Here is a list of the general variants of MOV, excluding segment registers:

Example:

```
MOV bx , 2
```

```
MOV ax , cx
```

Example:

‘A’ has ASCII code 65D (01000001B, 41H)

The following MOV instructions stores it in register BX:

- MOV bx , 65d
- MOV bx , 41h
- MOV bx , 01000001b
- MOV bx , ‘A’

All of the above are equivalent

Examples:

The following examples demonstrate compatibility between operands used with MOV instruction:

- | | |
|-------------------|---|
| • MOV ax , 2 | ✓ |
| • MOV ax , var | ✓ |
| • MOV var , ax | ✓ |
| • MOV 2 , ax | ✗ |
| • MOV var1 , var2 | ✗ |
| • MOV 5 , var | ✗ |

Overlapping Values**Example:**

```
.data
```

```
oneByte BYTE 78h
```

```
oneWord WORD 1234h
```

```
oneDword DWORD 12345678h
```

```
.code
```

```
Mov eax , 0 ; EAX = 00000000h
```

```
Mov al , oneByte ; EAX = 00000078h
```

```
Mov ax , oneWord ; EAX = 00001234h
```

```
Mov eax , oneDword ; EAX = 12345678h
```

```
Mov ax , 0 ; EAX = 12340000h
```

MOVZX Instruction

The MOVZX (MOV with zero-extend) instruction moves the contents and zero-extends the value to 16 or 32 bits. This instruction is only used with unsigned integers.

Syntax:

```
MOVZX reg32 , reg/mem8
```

```
MOVZX reg32 , reg/mem16
```

```
MOVZX reg16 , reg/mem8
```

The following examples use registers for all operands, showing all the size variations:

```
mov bx , 0A69Bh
```

```
movzx eax , bx ; EAX = 0000A69Bh
```

```
movzx edx , bl ; EDX = 0000009Bh
```

```
movzx cx , bl ; CX = 009Bh
```

BX = A69Bh

- BH = A6h (higher 8 bits)
- BL = 9Bh (lower 8 bits)

MOVSX Instruction

The MOVSX (MOV with sign extend) instruction moves the contents and sign extends the value to 16 or 32 bits. This instruction is only used with signed integers.

The following example uses registers for all operands, showing all the size variation:

```
Mov bx , 0A69Bh
```

```
Movsx eax , bx
```

```
Movsx edx , bl
```

```
Movsx cx , bl
```

INC Instruction

The INC instruction Takes an operand and adds 1 to it

Example:

```
MOV ax , 8           ; ax = 8
```

```
INC ax               ; ax = 9
```

DEC Instruction

The INC instruction Takes an operand and subtracts 1 from it

Example:

```
MOV ax , 10          ; ax = 10
```

```
DEC ax               ; ax = 9
```

XCHG Instruction

The XCHG (exchange data) instruction exchanges the contents of two operands. There are three variants:

XCHG does not accept immediate operands, use a register as a temporary container and combine MOV with XCHG:

```
mov ax , var1
```

```
xchg ax , var2
```

```
mov var1 , ax
```

ADD Instruction

The ADD instruction adds a source operand to a destination operand of the same size. Source is unchanged by the operation, and the sum is stored in the destination operand.

The syntax is:

```
ADD dest , source
```

SUB Instruction

The SUB instruction subtracts a source operand from a destination operand.

The syntax is:

```
SUB dest , source
```

NEG Instruction

The NEG (negate) instruction reverses the sign of a number by converting the number to its two's complement. The following operands are permitted:

NEG reg

NEG mem

Symbolic Constants

A symbolic constant (or symbol definition) is created by associating an identifier (a symbol) with an integer expression or some text. Symbols do not reserve storage. They are used only by the assembler when scanning a program, and they cannot change at runtime.

1) Equal-Sign (=) Directive

The equal sign (=) directive associates a symbol name with an integer expression. The syntax is

name = expression

Example:

COUNT = 500

mov eax, COUNT

mov eax, 500

Redefinitions: A symbol defined with can be redefined within the same program. The following example shows how the assembler evaluates COUNT as it changes value:

Count = 5

Mov al , count

Count = 10

Mov al , count

2) EQU Directive:

The EQU directive associates a symbolic name with an integer expression or some arbitrary text. Ordinarily, expression is a 32-bit integer value. When a program is assembled, all occurrences of name are replaced by expression during the assembler's preprocessor step.

name EQU expression

name EQU symbol

name EQU <text>

- Expression must be a valid integer expression
- Symbol is an existing symbol name, already defined with = or EQU.
- Any text may appear within the brackets < ... >

Example:

```
matrix1 EQU 10 * 10
```

```
matrix2 EQU <10 * 10>
```

```
.data
```

```
M1 WORD matrix1
```

```
M2 WORD matrix2
```

The assembler produces different data definitions for M1 and M2. The integer expression in matrix1 is evaluated and assigned to M1. On the other hand, the text in matrix2 is copied directly into the data definition for M2:

```
M1 WORD 100
```

```
M2 WORD 10*10
```

Example:

```
Rval = Xval * (Yval - Zval);
```

The following signed 32-bit variables will be used:


```
.data
```

```
Rval SDWORD ?
```

```
Xval SDWORD 26
```

```
Yval SDWORD 30
```

```
Zval SDWORD 40
```

```
.code
```

```
main PROC
```

```
    ; Rval = Xval * (Yval - Zval)
```

When translating an expression, evaluate each term separately and combine the terms at the end:

```
    mov eax, Yval    ; EAX = 30
```

```
    sub eax, Zval    ; EAX = 30 - 40 = -10
```

```
    imul eax, Xval   ; EAX = (Yval - Zval) * Xval = -10 * 26 = -260
```

```
    mov Rval, eax    ; store result
```

```
    mov eax, Rval    ; load result
```

```
    call WriteInt    ; print -260
```

```
    call Crlf
```

LAB TASKS

Task # 01:

Write a program that declares the following variables:

- A byte variable num1 initialized with 25.
- A word variable num2 initialized with 1200h.
- A doubleword variable num3 uninitialized.

Move the contents of num1 into AL, num2 into AX, and finally move both into num3 using `MOVZX` instruction.

Task # 02:

Declare a **signed byte** variable with value -50 and an **unsigned byte** variable with value 200.

- Use `movzx` to load the unsigned value into a 32-bit register.
- Use `movsx` to load the signed value into another 32-bit register.
- Display both results with `DumpRegs`.

Task # 03:

Declare two 16-bit variables: `num1 = 1234h`, `num2 = 5678h`.

- Load them into registers.
- Use `xchg` to swap their contents.
- Store the swapped results back into the variables.

Task # 04:

Declare three 32-bit variables:

Xval = 25, Yval = 15, Zval = 40

- Write a program to Compute : - (Xval + Yval - Zval)
- Use `add`, `sub`, and `neg` instructions step by step.
- Store the result in a variable `Rval`.

Task # 05:

Write a program to:

- Define a constant named MAX using `=` with value 100.
- Define another constant named MIN using `EQU` with value 10.
- Use these constants to initialize two variables and then add them together.
- Show the sum with `WriteInt`.